Cognitive Robot Intelligence Platform (CRIP)

A modular ROS 2 architecture for language-directed, self-improving autonomous navigation

Technical Report · 18 packages · 94 Python modules · 7,465 lines · 113 unit tests

1. Abstract

CRIP is a simulated cognitive robotics platform that converts natural-language commands into safe, grounded, closed-loop robot behaviour, and improves from its own logged experience. It integrates twelve subsystems — perception, a persistent semantic world model, an LLM task planner, vision-language scene assessment, classical (Nav2) and learned (PPO) navigation, trajectory prediction, semantic memory, a gated self-learning loop, an operator dashboard, an auditable safety supervisor, and fleet-wide sharing with explainable decisions — across 18 ROS 2 packages. Every AI component is backend-switchable with an offline fallback, so the system runs without API keys or GPU. This report documents the architecture, the verification performed, and — explicitly — the boundary between what is verified and what awaits integration testing.

2. Architecture

A mission flows top to bottom: a command enters via the dashboard; the LLM brain recalls similar past missions from semantic memory, produces a task plan, and grounds each referenced place against the world model (rejecting unknown references rather than hallucinating). Nav2 plans a path and emits velocity; perception, the VLM, and the trajectory predictor run continuously while moving. Every velocity command passes the safety supervisor before reaching the wheels. After the mission, the outcome is logged, feeding the self-learning loop, the fleet, and the explanation trace.

Design principles

- **Separation of logic from ROS.** Each subsystem's decision logic is a pure-Python core, unit-testable without a ROS graph; the ROS node is a thin wrapper. This is why 113 tests run in seconds with no simulator.
- **Backend-switchable AI.** LLM, VLM, and embeddings select between cloud (Anthropic), local (Ollama), and offline mock via environment variables, defaulting to mock.
- **Safety on the wire.** Sources publish to `/cmd_vel_raw`; the supervisor gates every command to `/cmd_vel`. Nothing reaches actuators unchecked.
- **Gated learning.** Retrained models are promoted only if they beat the current model on held-out data.

3. Verification performed

> **Verified in this build environment (no ROS/Gazebo available):** all 94 Python modules compile; 113 unit tests pass across 11 logic suites; real PPO and LSTM training executed with ONNX export matching the trained model to ~1e-8; the self-learning promotion gate correctly promoted an improved model and rejected

> a worse one; the safety engine's rule pipeline validated including the property that a safety veto is never re-accelerated by rate limiting; the FastAPI dashboard served and streamed state over WebSocket.

> **Not yet verified — requires a ROS 2 machine:** the integrated simulation has never been executed. Gazebo launch, Nav2 localization, the live multi-node graph, `rosidl` interface generation, and `ros_gz` bridge mappings are unverified. No demo video or real-hardware validation exists yet. The honest status is *logic verified, integration pending*.

4. Benchmark results

Logic-core throughput, measured with `benchmarks/run_benchmarks.py` (single core, no GPU). These measure component compute cost in isolation, not end-to-end mission latency. Control-path components must exceed the 20 Hz safety tick; all clear it by four to five orders of magnitude.

Component	Throughput	Latency	Budget
Safety engine (6-rule check)	199,617 /s	5.0 µs	≥ 20 Hz
Confidence fusion	375,415 /s	2.7 µs	≥ 20 Hz
Tracker update	553,735 /s	1.8 µs	≥ 20 Hz
Dashboard snapshot	522,560 /s	1.9 µs	≥ 5 Hz
Planner (mock)	438,720 /s	2.3 µs	per-mission
Memory recall (500 rec.)	55,342 /s	18.1 µs	per-query
Fleet share + merge	85,112 /s	11.7 µs	per-event
Knowledge graph query	32,879 /s	30.4 µs	per-query

5. Testing and CI

113 pytest tests cover the logic cores. A GitHub Actions pipeline (`.github/workflows/ci.yml`) runs two jobs on every push: a fast `logic-tests` job (compile + all unit suites), and a `ros-build` job that runs `colcon build` and `colcon test` inside the official `ros:jazzy` container. The first green `ros-build` will constitute the first integration verification.

6. Robustness features

Beyond the core milestones, three hardening layers were added: a **confidence manager** that fuses per-module confidence and gates speed (act / degrade / hold, capped by the weakest critical source); a **knowledge graph** giving the world model typed relations (`bottle on table in kitchen`) with transitive queries; and a **health monitor** classifying every module ok / slow / stale / missing with alerts to the dashboard.

7. Limitations and next steps

- **Integration untested.** The first task is `colcon build` on a ROS 2 Jazzy machine (or via CI) and resolving first-run `rosidl` / `ros_gz` issues.
- **Models are proof-level.** PPO and LSTM are trained as demonstrations, not to convergence; production quality needs GPU training.
- **AI backends default to mocks.** Real reasoning requires wiring Anthropic or a local Ollama model.
- **No manipulation or GPS.** The robot is a differential-drive navigation-and-inspection base by design.
- **Demo video pending** the first integrated run.

8. Reproducibility

Logic tests, benchmarks, and model training reproduce on any machine with Python 3.12 (no ROS). The full build reproduces via `docker compose up --build` on a ROS 2 machine, or automatically in CI. Full steps: `docs/REPRODUCIBILITY.md`.

Generated as project evidence. All performance figures are reproducible via the commands cited. The verified/unverified boundary is stated deliberately: this report describes a complete, tested architecture whose integration testing is the documented next phase, not a finished autonomous system.